Stock4event

Compras e estoque para eventos

Documentação técnica completa

Backend (Spring Boot) · Banco de dados (PostgreSQL) · API REST · App Mobile (Expo)

Do pedido ao quiosque: compras B2B e estoque de eventos em um só app.

Equipe

Luan Feitosa Santos — Back-end, líder

José Ítalo Silva Cordeiro Dantas — Banco de dados, testes

Marcelo Vitor Viana da Silva — Front-end, escopo

Leticia Viviane Pereira da Silva — Front-end, documentação

José Lucas Luiz da Silva — Fullstack

Instituição

UNINASSAU — ADS Caruaru

Projeto acadêmico · 2026

Professor responsável:

Antônio Almeida

Fontes analisadas

Backend: LuanSantos26/Stock4Event-BackEnd (branch leticia, commit 01371ae) + pasta database/

Frontend: LuanSantos26/Stock4Event-FrontEnd (branch leticia, commit 579d2bb)

Banco oficial: dump stock4event de 22/10/2026 · Identidade visual e apresentação padronizada

Versão 4 do documento · 22/10/2026 · Ajustes em subtelas, perfil da loja, foto de perfil e e-mail de boas-vindas

Sumário

1. Visão geral do projeto . . . 5
1.1 O que é o Stock4event . . . 5
1.2 Problema que resolve . . . 5
1.3 Os dois perfis de uso . . . 5
1.4 Equipe, instituição e repositórios . . . 5
2. Arquitetura geral . . . 7
2.1 Tecnologias . . . 7
2.2 Como as peças se conectam . . . 8
3. Como executar o projeto . . . 9
3.1 Pré-requisitos . . . 9
3.2 Banco e backend . . . 9
3.3 App mobile . . . 9
3.4 Conta de demonstração . . . 9
4. Backend (API REST) . . . 10
4.1 Organização em camadas . . . 10
4.2 Configuração (application.properties) . . . 10
4.3 Modelo de dados . . . 11
4.4 Referência da API (endpoints) . . . 12
4.5 Autenticação e segurança . . . 15
4.6 Regras de negócio . . . 15
4.7 Inicialização e dados iniciais . . . 17
4.8 Tratamento de erros . . . 18
5. Banco de dados oficial (PostgreSQL) . . . 19
5.1 Conexão . . . 19
5.2 Arquivos da pasta database/ . . . 19
5.3 Como montar o banco . . . 19
5.4 Diagrama entidade-relacionamento . . . 20
5.5 Tabelas e domínios . . . 21
5.6 Restrições importantes . . . 22
5.7 Dados iniciais do banco oficial . . . 22
5.8 Consultas úteis . . . 23
5.9 Backup, restauração e a troca de 21/10/2026 . . . 23
6. Identidade visual . . . 24
6.1 Arquivos originais da marca . . . 24

6.2 Paleta de cores . . . 24
6.3 Versões do logo e quando usar . . . 24
6.4 Como a marca entrou no código . . . 25
6.5 Telas com a nova identidade . . . 25
6.6 Mensagens da marca . . . 27
7. Frontend (aplicativo mobile) . . . 28
7.1 Estrutura de pastas . . . 28
7.2 Inicialização do app . . . 28
7.3 Conexão com o backend . . . 28
7.4 Autenticação e sessão no app . . . 29
7.5 Navegação . . . 29
7.6 Contextos (estado global) . . . 30
7.7 Telas . . . 30
7.8 Componentes reutilizáveis . . . 31
7.9 Regras no app e testes . . . 32
7.10 Identidade visual no código . . . 32
7.11 Dados salvos no aparelho (AsyncStorage) . . . 32
8. Fluxos de ponta a ponta . . . 33
8.1 Cadastro e login . . . 33
8.2 Compra no marketplace . . . 33
8.3 Fornecedor gerencia pedidos recebidos . . . 33
8.4 Cliente monta quiosques com o que comprou . . . 33
8.5 Cadastro de produto com foto . . . 34
9. Estado atual, limitações e pontos de atenção . . . 35
9.1 O que está integrado de verdade . . . 35
9.2 O que ainda é demonstração/mock . . . 35
9.3 Situação dos repositórios . . . 35
9.4 Riscos técnicos . . . 35
9.5 Documentação existente que está desatualizada . . . 36
9.6 Próximos passos (roadmap da apresentação) . . . 36
10. Novidades da versão 3 (22/10/2026) . . . 37
10.1 Segurança: JWT em todas as rotas . . . 37
10.2 PDV (ponto de venda) . . . 37
10.3 Novas distribuidoras e catálogos . . . 37
10.4 Vitrine entre contas . . . 38
10.5 Correções . . . 38

10.6 Banco de dados atualizado . . . 38
10.7 Testes automatizados . . . 39
11. Novidades da versão 4 (22/10/2026) . . . 40
11.1 Ajustes em subtelas . . . 40
11.2 Perfil da loja e foto de perfil . . . 40
11.3 E-mail de boas-vindas . . . 40
11.4 Configuração do envio . . . 41
11.5 Validação e confirmação de e-mail . . . 42
11.6 Banco de dados atualizado . . . 42
11.7 Testes automatizados . . . 42
Apêndice — Glossário . . . 43
Histórico de versões do documento . . . 43

1. Visão geral do projeto

1.1 O que é o Stock4event

O **Stock4event** (antes chamado *QuickStock*, renomeado em 20/10/2026) é uma plataforma de **gestão de estoque e compras B2B para o setor de bebidas e eventos**. Ela junta, em um único aplicativo, duas coisas que normalmente ficam separadas: a **gestão interna** de produtos e pontos de venda (quiosques/barracas em eventos) e as **compras externas** junto a distribuidoras, com uma experiência parecida com a de apps de delivery (vitrine, sacola, checkout e acompanhamento do pedido em tempo real).

O sistema é formado por três peças que conversam entre si:

- **App mobile** — feito em Expo / React Native com TypeScript. É o que o usuário vê e usa.
- **Backend (API REST)** — feito em Java 17 com Spring Boot 3.4. Recebe as requisições do app, aplica as regras de negócio (estoque, pedidos, pagamentos, financeiro) e grava no banco.
- **Banco de dados PostgreSQL** — guarda tudo de forma permanente (banco **stock4event**).

1.2 Problema que resolve

Comerciantes de bebidas (bares, mercados, quem monta barracas em festas e eventos) precisam controlar estoque em vários pontos de venda e repor mercadoria com distribuidoras. Normalmente isso é feito por telefone, WhatsApp e planilhas. O Stock4event centraliza: o comerciante compra pelo app, acompanha a entrega, e quando o pedido chega o estoque dele é atualizado automaticamente — e daí ele pode dividir esse estoque entre seus quiosques.

1.3 Os dois perfis de uso

Na hora do cadastro o usuário escolhe um perfil. Cada perfil tem telas, barra inferior e cor próprias:

Perfil	Quem é	Cor	Abas da barra inferior	Tipo da empresa no banco
Cliente	Comerciante que compra bebidas de distribuidoras e revende em quiosques	Âmbar #FFAD19	Início, Produtos, Carrinho, Pedidos, Quiosques, Perfil	`CLIENTE`
Fornecedor	Distribuidora que vende, gerencia catálogo, quiosques e vendas	Azul-marinho #092D5E	Início, Quiosques, Produtos, Vendas, Ajustes	`DISTRIBUIDOR`
Cliente e fornecedor	Empresa que faz os dois papéis	—	Fica na visão de Fornecedor (só a conta demo alterna)	`AMBOS`

1.4 Equipe, instituição e repositórios

Projeto acadêmico de 2026 do curso de Análise e Desenvolvimento de Sistemas (ADS) da **UNINASSAU Caruaru**, sob responsabilidade do professor **Antônio Almeida**. Papéis da equipe, conforme a apresentação oficial do projeto:

Integrante	Papel no projeto
Luan Feitosa Santos	Back-end · líder da equipe

Integrante	Papel no projeto
José Ítalo Silva Cordeiro Dantas	Banco de dados · testes
Marcelo Vitor Viana da Silva	Front-end · escopo
Leticia Viviane Pereira da Silva	Front-end · documentação
José Lucas Luiz da Silva	Fullstack

Repositório	Conteúdo	Situação local
`github.com/LuanSantos26/QuickStock-BackEnd`	Backend + pasta **database/** com o banco oficial (remote *luan*)	C:\Dev\Code\Faculdade\Stock4event-BackEnd, branch **integracao-luan**
`github.com/itsmelvys/Back-End`	Backend — fork pessoal (remote *origin*)	origin/main está atrás
`github.com/LuanSantos26/Stock4Event-FrontEnd`	App mobile — repositório da equipe. Renomeado em 21/10/2026 (antes: LuanSantos26/Mobile)	C:\Dev\Code\Faculdade\Stock4Event-FrontEnd, branch **leticia**
`github.com/marcelo435/Mobile`	Fork do Marcelo (remote *marcelo*)	Já mesclado na branch leticia
Faculdade\Mobile e Documents\projeto\QuickStock-BackEnd	Clones antigos	Mantidos só como backup — não usar

Nota: O nome do repositório do backend no GitHub ainda é **QuickStock-BackEnd**; só o código foi renomeado para Stock4event.

2. Arquitetura geral

A arquitetura é a clássica **cliente–servidor em camadas**. O app nunca fala direto com o banco: toda leitura e escrita passa pela API REST, que é a única dona dos dados. O desenho abaixo mostra as peças e por onde os dados passam.

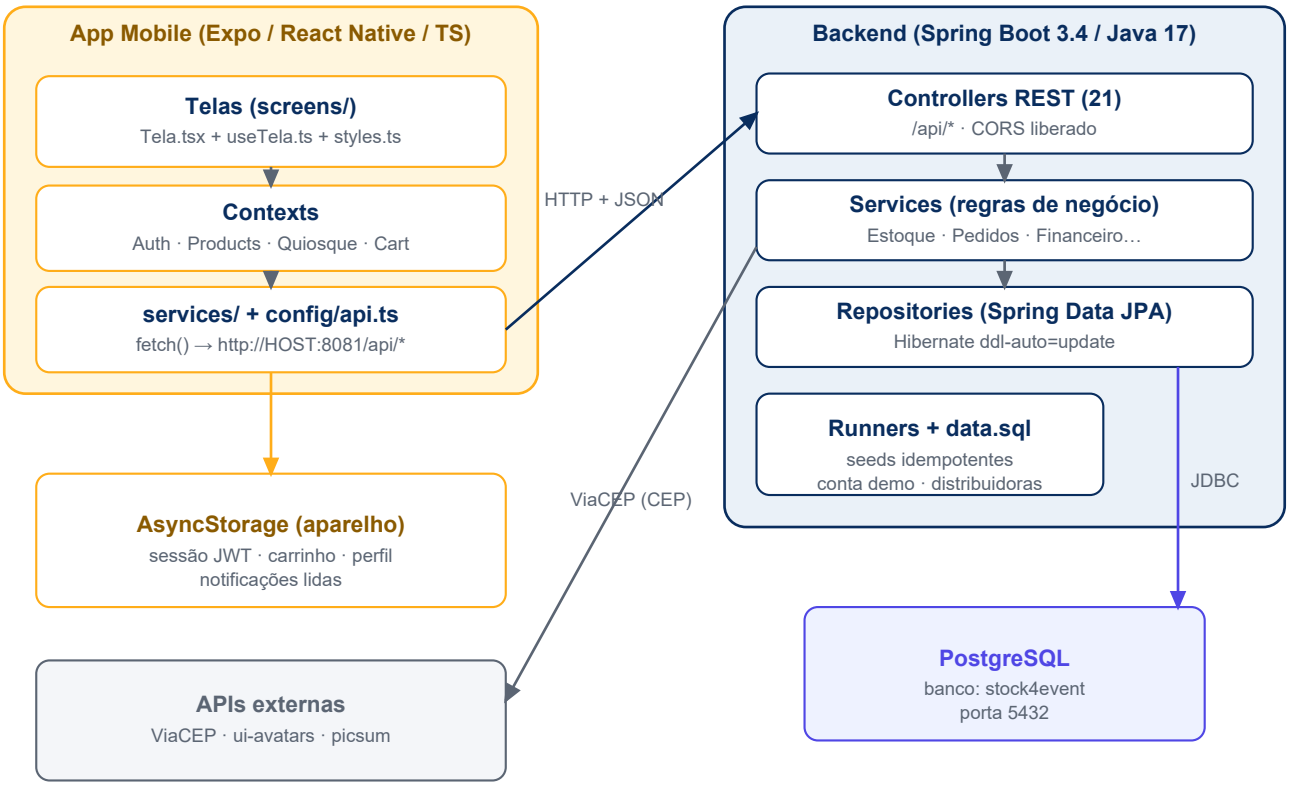


Figura 1 — Arquitetura do Stock4event (mobile, backend, banco e serviços externos).

2.1 Tecnologias

Camada	Tecnologia	Versão	Para que serve
Mobile	Expo	~54.0	Ferramentas de build/execução do React Native (Expo Go, web, Android, iOS)
Mobile	React Native / React	0.81.5 / 19.1	Biblioteca de interface
Mobile	TypeScript	~5.9 (strict)	Tipagem estática do código do app
Mobile	React Navigation (native-stack)	7.x	Navegação entre telas
Mobile	AsyncStorage	2.2	Armazenamento local (sessão, carrinho, preferências)
Mobile	expo-image-picker / image-manipulator	17 / 14	Escolher e redimensionar foto de produto
Mobile	Jest + jest-expo	29 / 54	Testes unitários
Backend	Java	17	Linguagem
Backend	Spring Boot (web, data-jpa, validation)	3.4.0	Framework da API, persistência e validação
Backend	Hibernate / JPA	(via Spring)	Mapeia classes Java para tabelas
Backend	Lombok	1.18.36	Gera getters/setters (@Data)

Camada	Tecnologia	Versão	Para que serve
Backend	spring-security-crypto (BCrypt)	(via Spring)	Hash de senhas
Backend	jjwt	0.12.6	Geração e validação de tokens JWT
Backend	spring-boot-starter-mail (Jakarta Mail)	3.4.0	Envio do e-mail de boas-vindas por SMTP
Backend	springdoc-openapi	2.8.6	Swagger UI em /swagger
Banco	PostgreSQL	18 (local) · dump gerado no 16	Banco relacional — ver capítulo 5

2.2 Como as peças se conectam

- **Protocolo:** HTTP com corpo em JSON. Toda rota da API começa com `/api/`.
- **Porta da API:** `8081` (definida em `server.port`). O Expo roda na porta `8082` para não conflitar.
- **Endereço do backend visto pelo app:** calculado em `src/config/api.ts`: usa o IP da máquina que roda o Expo (lido de `Constants.expoConfig.hostUri`), ou `10.0.2.2` no emulador Android, ou `localhost`.
- **Banco:** o backend conecta via JDBC em `jdbc:postgresql://localhost:5432/stock4event` (usuário postgres / senha admin).
- **Imagens:** o backend serve arquivos em `/uploads/**` (pasta local `uploads/` + imagens empacotadas em `static/uploads`). O app transforma caminhos relativos em URL completa com `getImageUrl()`.
- **CORS:** liberado para qualquer origem (`WebConfig`), o que permite rodar o app no navegador (Expo web).
- **Autenticação:** login devolve um **JWT**; o app guarda no AsyncStorage e usa em `GET /api/usuarios/me` para restaurar a sessão.

> **Dica:** Num celular físico com Expo Go, o celular e o computador que roda o backend precisam estar na **mesma rede Wi-Fi**, e o firewall do Windows precisa permitir conexões na porta 8081.

3. Como executar o projeto

3.1 Pré-requisitos

- Java 17+ (o projeto usa o Maven Wrapper, então não precisa instalar Maven).
- PostgreSQL rodando em localhost:5432 com usuário **postgres** e senha **admin** (ou ajuste o application.properties).
- Node.js LTS + npm; Expo Go no celular ou um emulador Android/iOS.

3.2 Banco e backend

Há dois caminhos para ter o banco pronto, e os dois chegam ao mesmo resultado (detalhes no capítulo 5):

```
# Caminho A — deixar o backend criar tudo
CREATE DATABASE stock4event;          -- no psql / pgAdmin
cd C:\Dev\Code\Faculdade\Stock4event-BackEnd
mvnw.cmd spring-boot:run              -- Hibernate cria as tabelas, data.sql e runners populam

# Caminho B — restaurar o banco oficial com os scripts (Git Bash)
export PATH="/c/Program Files/PostgreSQL/18/bin:$PATH"
export PGPASSWORD=admin
cd database
./restore_stock4event.sh stock4event_full.sql      # ou ./build_from_scratch.sh

# API:      http://localhost:8081
# Swagger:  http://localhost:8081/swagger
```

3.3 App mobile

```
cd C:\Dev\Code\Faculdade\Stock4Event-FrontEnd
npm install
npm start          # Expo na porta 8082 (a = Android, i = iOS, w = web, ou QR Code)
npm test           # testes unitários (Jest)
```

3.4 Conta de demonstração

Campo	Valor
E-mail	`demo@stock4event.com`
Senha	`demo1234`
Empresa	Mercado Demo (tipo CLIENTE no banco; perfil de uso “ClienteFornecedor”)
Já vem com	Endereço em Caruaru-PE, cartão Visa final 4242, chave PIX (CNPJ) e formas de pagamento
Particularidade	É a **única** conta que pode alternar entre visão Cliente e Fornecedor (botão no fim da barra inferior ou Ajustes → Alternar visão)

Para a tela de login vir preenchida, existe um `.env.local` (fora do git) com `EXPO_PUBLIC_LOGIN_EMAIL` e `EXPO_PUBLIC_LOGIN_SENHA`.

4. Backend (API REST)

4.1 Organização em camadas

O backend segue o padrão **Controller** → **Service** → **Repository** → **Entity**, todo dentro do pacote `com.stock4event.backend`:

Pacote	Qtd.	Responsabilidade	Exemplo
`controller`	21	Recebe a requisição HTTP, lê parâmetros/corpo e devolve a resposta (status + JSON)	`SolicitacaoCompraController`
`service`	20	Regras de negócio, validações e transações (@Transactional)	`SolicitacaoCompraService`
`repository`	19	Interfaces Spring Data JPA; consultas geradas pelo nome do método	`findByEmpresaIdAndAtivo(...)`
`entity`	19	Classes mapeadas para tabelas (@Entity, @Table)	`Produto → produtos`
`dto`	43	Objetos de entrada/saída da API (não expõem a entidade inteira)	`ProdutoResponseDTO`
`config`	7	CORS, arquivos estáticos, seeds de dados (ApplicationRunner) e configuração do e-mail	`WebConfig, EmailConfig`
`exception`	2	Exceção de negócio e tratador global de erros	`CadastroException`

Um exemplo do caminho de uma requisição: o app chama `POST /api/solicitacoes-compra` → `SolicitacaoCompraController.criar()` → `SolicitacaoCompraService.criar()` (valida, debita estoque, cria pedido e pagamento) → vários `Repository.save()` → Hibernate gera os `INSERT/UPDATE` no PostgreSQL → o controller devolve um `SolicitacaoCompraResponseDTO` em JSON.

4.2 Configuração (application.properties)

Propriedade	Valor	Significado
`spring.datasource.url`	`jdbc:postgresql://localhost:5432/stock4event`	Banco usado
`spring.datasource.username / password`	`postgres / admin`	Credenciais locais
`spring.jpa.hibernate.ddl-auto`	`update`	Cria/atualiza tabelas a partir das entidades
`spring.jpa.show-sql`	`true`	Mostra o SQL no console (útil para depurar)
`spring.sql.init.mode`	`always`	Executa o data.sql a cada inicialização
`spring.jpa.defer-datasource-initialization`	`true`	Roda o data.sql depois que o Hibernate cria as tabelas
`server.port`	`8081`	Porta da API
`jwt.secret / jwt.expiration-ms`	`… / 86400000`	Chave de assinatura e validade do token (24 h)
`spring.servlet.multipart.max-file-size`	`10MB`	Tamanho máximo de upload
`upload.dir`	`uploads/produtos`	Onde as fotos de produto são gravadas; logo/capa e foto de perfil vão para as pastas irmãs `uploads/empresas` e `uploads/usuarios`

Propriedade	Valor	Significado
`stock4event.email.habilitado`	`${STOCK4EVENT_EMAIL_HABILITADO:false}`	Liga o e-mail de boas-vindas (desligado por padrão)
`stock4event.email.remetente / url-app`	`${STOCK4EVENT_EMAIL_REMETENTE} / ${STOCK4EVENT_APP_URL}`	Remetente e endereço do botão “Acessar minha conta”
`spring.mail.*`	`SPRING_MAIL_HOST, _PORT, _USERNAME, _PASSWORD`	Servidor e credenciais SMTP, só por variável de ambiente
`springdoc.swagger-ui.path`	`/swagger`	Atalho para a documentação interativa
`stock4event.pedidos.status-automatico`	`true`	Pedidos avançam sozinhos de status (modo demonstração)
`stock4event.cnpj.validacao-receita`	(padrão true)	Usada pelo CnpjConsultaService (ainda não integrado — ver cap. 9)

4.3 Modelo de dados

> **Nota:** Este item resume as entidades do ponto de vista do código. O banco oficial, com diagrama ER, scripts, restrições e dados iniciais, está no **capítulo 5**.

São **19 tabelas**. A **empresa** é o centro de tudo: usuários, produtos, endereços, cartões e pedidos pertencem a uma empresa. Todas as chaves primárias são `BIGINT IDENTITY` (auto incremento).

Tabela	Entidade	Campos principais	Relacionamentos
`perfis`	Perfil	nome (único), descricao	1 perfil → N usuários. Seed: “Admin”
`empresas`	Empresa	nome, cnpj (único; aceita CPF), telefone, tipo, descricao, logo_url, capa_url (editáveis no Perfil da loja), criado_em	Raiz de quase tudo
`usuarios`	Usuario	nome, email (único), senha_hash, ativo, perfil_uso, cpf, foto_url	N:1 empresa, N:1 perfil
`produtos`	Produto	nome, preco_venda, unidade, descricao, imagem_url, ativo, estoque, codigo (único), codigo_origem	N:1 empresa
`eventos`	Evento	nome, data_inicio, data_fim, status	N:1 empresa
`barracas`	Barraca (quiosque)	nome, ativa	N:1 evento, N:1 usuário responsável
`estoque_barraca`	EstoqueBarraca	quantidade, atualizado_em — UNIQUE(barraca_id, produto_id)	N:1 barraca, N:1 produto
`solicitacoes_compra`	SolicitacaoCompra	valor_total, status, metodo_pagamento, cópia do endereço, taxa_entrega, estoque_comprador_creditado	empresa compradora, empresa fornecedora, usuário, pedido
`itens_solicitacao_compra`	ItemSolicitacaoCompra	quantidade, preco_unitario, subtotal	N:1 solicitação, N:1 produto
`pedido`	Pedido	tipo (pdv/marketplace), status, valor_total, endereço, metodo_pagamento, taxa_entrega	barraca, operador, empresas, endereço
`itens_pedido`	ItemPedido	quantidade, preco_unitario, subtotal	N:1 pedido, N:1 produto
`pagamentos`	Pagamento	metodo, valor, status, referencia_pagamento	N:1 pedido

Tabela	Entidade	Campos principais	Relacionamentos
`enderecos_entrega`	EnderecoEntrega	apelido, logradouro, numero, complemento, bairro, cidade, uf, cep, principal	N:1 empresa
`formas_pagamento_salvas`	FormaPagamentoSalva	tipo (pix/credito/debito/dinheiro), apelido, principal	N:1 empresa
`cartoes_pagamento_salvos`	CartaoPagamentoSalvo	tipo, bandeira, ultimos_digitos, numero_mascarado, validade, titular (sem CVV)	N:1 empresa
`chaves_pix_salvas`	ChavePixSalva	apelido, tipo_chave, chave_mascarada, ultimos_digitos	N:1 empresa
`caminhoneiros`	Caminhoneiro	nome, veiculo, rota, status	N:1 empresa; 1:N entregas
`entregas_logistica`	EntregaLogistica	nome_cliente, cidade, rota, status	N:1 caminhoneiro
`email_boas_vindas`	EmailBoasVindas	destinatario, tipo, status, tentativas, ultimo_erro, proxima_tentativa_em, enviado_em	usuario_id único (ligação lógica, sem FK)

Valores importantes de campos “tipo” e “status”

Campo	Valores	Observação
`empresas.tipo`	`CLIENTE, DISTRIBUIDOR, AMBOS` (legado: COMPRADOR, INATIVO)	Só DISTRIBUIDOR e AMBOS aparecem no marketplace como fornecedores
`usuarios.perfil_uso`	`Cliente, Fornecedor, ClienteFornecedor`	Define quais telas o app mostra
`solicitacoes_compra.status`	`aguardando_liberacao, em_rota, entregue, cancelada`	Ver máquina de estados em 4.6.3
`pagamentos.status`	`aprovado, pendente, estornado, cancelado`	PIX/crédito/débito nascem “aprovado”; dinheiro nasce “pendente”
`pedido.tipo`	`pdv, marketplace`	Pedidos de compra B2B usam “marketplace”
`produtos.codigo`	`EMP-{empresaId}-{0001}`	Gerado automaticamente; `codigo_origem` liga o produto do comprador ao do fornecedor

> **Nota:** Existem **duas representações** da mesma compra: a `solicitacoes_compra` (visão do comprador, com o fluxo de status) e o `pedido` + `itens_pedido` + `pagamentos` (visão de venda/financeiro). O serviço mantém as duas sincronizadas: toda mudança de status é gravada nas duas.

4.4 Referência da API (endpoints)

Todas as respostas são JSON. Em caso de erro, o corpo segue o formato `{"erro": "mensagem"}`. Parâmetros com **?** são *query string*; **{id}** é parte do caminho. Os endpoints marcados com **APP** são usados pelo app mobile.

Autenticação, usuários e cadastro

Método	Rota	Descrição
POST	`/api/cadastro` APP	Cadastro unificado: cria **empresa + usuário admin** numa transação. Corpo: `{empresa:{nome,cnpj,telefone}, usuario:{nome,email,senha,cpf}, perfil}`. 409 se e-mail ou CNPJ/CPF já existir. Depois de salvo, agenda o e-mail de boas-vindas (cap. 11)
POST	`/api/usuarios/login` APP	Body `{email, senha}` → `{token, expiresIn, usuario}`. 401 se inválido ou inativo
GET	`/api/usuarios/me` APP	Header `Authorization: Bearer <token>` → usuário atual (restaura a sessão)
POST	`/api/usuarios/recuperar-senha` APP	Body `{email, novaSenha}` (mín. 6 caracteres). Troca a senha direto, sem e-mail de confirmação
PUT	`/api/usuarios/{id}` APP	O próprio usuário atualiza nome, e-mail, senha e foto (`fotoUrl`); campos ausentes não mudam, texto vazio remove a foto
POST	`/api/usuarios/upload` APP	multipart `file` → `{"imagemUrl": "/uploads/usuarios/<uuid>.jpg"}` (foto de perfil)
POST	`/api/usuarios/register`	Registro antigo (exige perfilId e empresaId já existentes)
GET	`/api/usuarios, /{id}, /empresa/{empresaId}`	Listagens
DELETE	`/api/usuarios/{id}`	Desativa (ativo = 0), não apaga
CRUD	`/api/perfis`	GET lista/{id}, POST, PUT, DELETE de perfis
PUT	`/api/empresas/{id}` APP	Nome, CNPJ, telefone e perfil da loja (`descricao, logoUrl, capaUrl`); campos ausentes não mudam. GET lista/{id}; POST e DELETE recusados (403)
POST	`/api/empresas/upload` APP	multipart `file` → `{"imagemUrl": "/uploads/empresas/<uuid>.jpg"}` (foto e capa da loja)

Produtos e catálogo

Método	Rota	Descrição
GET	`/api/produtos?empresaId=` APP	Produtos ativos da empresa
GET	`/api/produtos/{id}` APP	Um produto
POST	`/api/produtos/upload` APP	multipart com campo `file` (JPEG/PNG/WebP/HEIC, até 10 MB) → `{"imagemUrl": "/uploads/produtos/<uuid>.jpg"}`
POST	`/api/produtos` APP	Cria produto (`nome, precoVenda, unidade, descricao, imagemUrl, estoque, empresaId`); o código é gerado
PUT	`/api/produtos/{id}` APP	Edita; valida que o produto é da empresa informada
DELETE	`/api/produtos/{id}?empresaId=` APP	Desativação lógica (ativo = 0)

Marketplace e pedidos B2B

Método	Rota	Descrição
GET	`/api/marketplace/fornecedores?empresaCompradoraId=` APP	Empresas DISTRIBUIDOR/AMBOS (menos a própria), com total de produtos
GET	`/api/marketplace/fornecedores/{id}/produtos` APP	Catálogo ativo de um fornecedor

Método	Rota	Descrição
POST	`/api/solicitacoes-compra` APP	Cria compra: `{empresaCompradoraId, empresaFornecedoraId, usuarioId, metodoPagamento, enderecoEntregaId, taxaEntrega, pagamentoReferencia, itens:[{produtoId, quantidade}]}`
GET	`/api/solicitacoes-compra?empresaCompradoraId=` APP	Compras feitas pela empresa (já com status atualizado)
GET	`/api/solicitacoes-compra/{id}?empresaCompradoraId=` APP	Detalhe + etapas da linha do tempo + previsão de entrega (usado no polling)
GET	`/api/solicitacoes-compra/recebidas?empresaFornecedoraId=` APP	Pedidos que o fornecedor recebeu
PATCH	`/api/solicitacoes-compra/{id}/status` APP	Fornecedor muda status. Body `{empresaFornecedoraId, status}` (em_rota | entregue | cancelada)
PATCH	`/api/solicitacoes-compra/{id}/cancelar?empresaCompradoraId=` APP	Comprador cancela (só antes de sair para entrega)

Quiosques (barracas), eventos e estoque por quiosque

Método	Rota	Descrição
GET	`/api/barracas?empresaId=` APP	Quiosques ativos da empresa, com itens de estoque
POST / PUT	`/api/barracas, /api/barracas/{id}` APP	Cria/edita: `{nome, empresaId, responsavelId, itens:[{produtoId, quantidade}]}`
PUT	`/api/barracas/{id}/estoque`	Sincroniza só o estoque do quiosque
DELETE	`/api/barracas/{id}?empresaId=` APP	Desativa (ativa = 0)
CRUD	`/api/eventos`	Eventos (filtros empresaId/status). Barracas criadas pelo app caem num evento padrão “Operação principal”
CRUD	`/api/estoque-barraca`	Linhas de estoque; `PATCH /{id}/quantidade` ajusta; `GET /barraca/{b}/produto/{p}` consulta saldo

Endereços e pagamento

Método	Rota	Descrição
GET	`/api/enderecos?empresaId=` APP	Endereços da empresa
GET	`/api/enderecos/cep/{cep}` APP	Consulta o CEP no ViaCEP (pelo backend) e devolve logradouro/bairro/cidade/UF
POST / PUT / DELETE	`/api/enderecos[/{id}]` APP	CRUD; ao remover, pedidos antigos mantêm a cópia do endereço
PATCH	`/api/enderecos/{id}/principal?empresaId=` APP	Define o endereço principal
GET/POST/DELETE	`/api/formas-pagamento` APP	Formas salvas (pix, credito, debito, dinheiro); `PATCH /{id}/principal`
GET/POST/DELETE	`/api/cartoes-pagamento` APP	Cartões salvos (só dados mascarados; CVV nunca é guardado). GET aceita `tipo`
GET/POST/DELETE	`/api/chaves-pix` APP	Chaves PIX salvas (mascaradas)

Método	Rota	Descrição
CRUD	`/api/pedidos, /api/itens-pedido, /api/pagamentos`	Acesso direto às tabelas de pedido/PDV (não usado pelo app atual)

Financeiro, notificações e logística

Método	Rota	Descrição
GET	`/api/financeiro/resumo?empresaCompradoraId=` APP	Últimos 6 meses: compras, vendas, lucro, margem, pedidos por mês e formas de pagamento
GET	`/api/financeiro/stock-dia?empresaCompradoraId=` APP	Movimentação de estoque do dia (por hora)
GET	`/api/notificacoes?empresaCompradoraId=` APP	Notificações geradas na hora (não há tabela)
GET / POST	`/api/caminhoneiros`	Lista / cadastra motorista (opcionalmente já com uma entrega)
GET	`/api/logistica?empresaId=`	Resumo da entrega ativa com 5 etapas

4.5 Autenticação e segurança

O login é feito em `UsuarioController.login()`: busca o usuário pelo e-mail (sem diferenciar maiúsculas), confere se está ativo e compara a senha com o hash usando `BCrypt.checkpw`. Se bater, o `JwtService` gera um token HMAC-SHA com:

- `sub` = id do usuário; `email` como claim extra;
- validade de 24 h (`jwt.expiration-ms`);
- assinatura com a chave `jwt.secret`.

O app guarda o token e o envia em `GET /api/usuarios/me` ao reabrir. As senhas nunca são salvas em texto: o cadastro gera o hash com `BCrypt.hashpw(senha, gensalt())`.

> **Dica:** Desde a versão 3, **todas as rotas exigem o token** (Spring Security + `JwtAuthenticationFilter`). Só login, cadastro, recuperação de senha e as imagens em `/uploads/**` são públicos. A empresa é lida do token (`AcessoService`), nunca do `empresaId` enviado pelo app. Detalhes no capítulo 10.

4.6 Regras de negócio

4.6.1 Cadastro de conta (CadastroService)

- Normaliza o e-mail (trim + minúsculas) e valida o perfil (Cliente, Fornecedor ou ClienteFornecedor; padrão Fornecedor).
- Recusa com **409** se o e-mail já existir ou se o CNPJ/CPF já existir (a mensagem diz “CPF” se o documento tiver 11 dígitos).
- Cria a empresa com o tipo derivado do perfil: Cliente → CLIENTE, ClienteFornecedor → AMBOS, Fornecedor → DISTRIBUIDOR.
- Cria o usuário com perfil de acesso **Admin** (procura Admin, Gestor ou Gerente), senha em BCrypt e CPF só com dígitos.
- Tudo em uma única transação: se algo falhar, nada é gravado.

4.6.2 Compra no marketplace (SolicitacaoCompraService.criar)

É a regra mais importante do sistema. Passo a passo:

- Valida: há itens; compradora e fornecedora existem; a fornecedora é DISTRIBUIDOR/AMBOS; não é a própria empresa; o usuário pertence à compradora; forma de pagamento é pix, credito, debito ou dinheiro; o endereço pertence à compradora.
- Cria a solicitação com status `aguardando_liberacao` e copia o endereço (assim apagar o endereço depois não afeta o histórico).
- Para cada item: confere que o produto é do fornecedor e que há estoque; **debita o estoque do fornecedor na hora**; grava o item com o preço atual.
- Total = soma dos subtotais + taxa de entrega.
- Cria em paralelo um `Pedido` do tipo *marketplace* com os mesmos itens e um `Pagamento` simulado (pix/crédito/débito = aprovado; dinheiro = pendente).
- Se faltar estoque em qualquer item, a exceção desfaz tudo (@Transactional).

4.6.3 Ciclo de vida do pedido

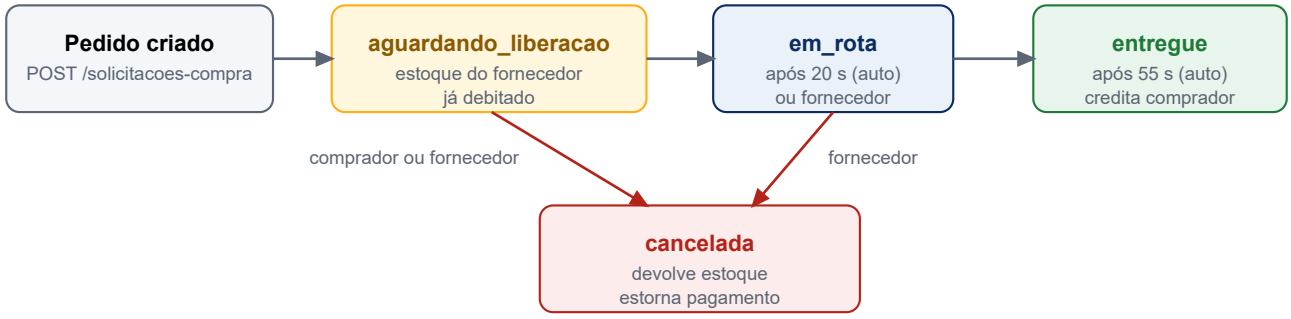


Figura 2 — Máquina de estados da solicitação de compra.

Transição	Quem pode	Efeito colateral
aguardando_liberacao → em_rota	Fornecedor (PATCH /status) ou automático após 20 s	—
em_rota → entregue	Fornecedor ou automático após 55 s	**Credita o estoque do comprador** (uma única vez, controlado por `estoque_comprador_creditado`)
aguardando_liberacao → cancelada	Comprador (PATCH /cancelar) ou fornecedor	Devolve o estoque ao fornecedor; pagamento aprovado vira "estornado", os outros "cancelado"
em_rota → cancelada	Somente o fornecedor	Mesmo efeito acima

Modo demonstração: com `stock4event.pedidos.status-automatico=true`, não existe um agendador rodando. O avanço é *preguiçoso*: toda vez que o pedido é lido (listar, buscar, recebidas), o método `atualizarStatusDemonstracao()` compara a hora de criação com a hora atual e aplica as transições vencidas. Como o app faz polling a cada 5 s, o usuário vê o status mudar ao vivo.

A resposta também traz **etapas** para a linha do tempo (Pedido confirmado → Preparando → Saiu para entrega → Entregue), um rótulo amigável e uma **previsão de entrega** em minutos.

4.6.4 Estoque do comprador (EstoqueProdutoService)

Quando um pedido é entregue, para cada item o serviço procura na empresa compradora um produto com `codigo_origem` igual ao código do produto do fornecedor. Se não existir, cria uma cópia (mesmo nome, preço, imagem, unidade) com código novo `EMP-{id}-NNNN` e estoque 0. Depois soma a quantidade recebida. É isso que permite ao Cliente montar quiosques com o que comprou.

4.6.5 Quiosques (BarracaService)

- Um quiosque pertence a um **evento**. Se a empresa não tem evento com status “ativo”, o serviço cria um evento padrão “Operação principal”.
- O responsável precisa ser um usuário da mesma empresa.
- O estoque do quiosque fica em `estoque_barraca` (uma linha por produto); salvar o quiosque **sincroniza** essas linhas.
- Excluir é desativar (`ativa = 0`).

4.6.6 Financeiro (FinanceiroService)

Calcula, para os últimos 6 meses, compras (solicitações onde a empresa comprou), vendas (pedidos onde a empresa vendeu), lucro = vendas − compras, margem do mês atual, médias e contagem de pedidos, além da distribuição por forma de pagamento. Ignora registros marcados com a observação `SEED_FINANCEIRO_DEMO`. O endpoint `stock-dia` monta a movimentação de estoque do dia por hora.

4.6.7 Notificações (NotificacaoService)

Não existe tabela de notificações: a lista é **montada na hora**. Ela inclui as 8 compras mais recentes (com título conforme o status). Contas com menos de 7 dias recebem só uma mensagem de boas-vindas; contas mais antigas recebem também promoções e ofertas **fixas no código** associadas às distribuidoras cadastradas. O estado “lida” é controlado só no aparelho.

4.6.8 Outras regras

- **Upload de imagem** (`ProdutoUploadService`): valida tipo e tamanho, salva com nome UUID em `uploads/produtos`, `uploads/empresas` ou `uploads/usuarios` e devolve a URL relativa. Logo, capa e foto só aceitam caminhos `/uploads/...` do próprio servidor (`caminhoImagemValido`); links externos são ignorados.
- **CEP** (`CepConsultaService`): exige 8 dígitos, consulta `viacep.com.br` pelo `RestClient` do Spring; erro de rede vira 502.
- **Endereços**: só um pode ser principal; remover desvincula de pedidos (que guardam cópia).
- **Formas de pagamento / cartões / PIX**: sempre validam que o registro pertence à empresa informada antes de remover.
- **Logística**: cadastrar um caminhoneiro com entrega o coloca em “Carregando”; o resumo calcula 5 etapas (Confirmado → Separação → Carregamento → Em trânsito → Entrega prevista).

4.7 Inicialização e dados iniciais

Ao subir, o backend executa nesta ordem (todos idempotentes — reiniciar não duplica nada):

Ordem	O quê	Faz
1	Hibernate (`ddl-auto=update`)	Cria tabelas/colunas que faltarem a partir das entidades

Ordem	O quê	Faz
2	`data.sql`	Perfil Admin, migrações `ADD COLUMN IF NOT EXISTS`, 3 distribuidoras e 15 produtos com código `MKT-…`, estoque inicial
3	`DemoSeedRunner` (@Order 80)	Conta demo com endereço, cartão e PIX
4	`FormaPagamentoSeedRunner` (@Order 91)	Formas de pagamento padrão
5	`ProdutoCatalogoFixRunner`	Garante o catálogo oficial: troca códigos provisórios `LEG-<id>` por `MKT-…`, corrige preços e imagens
6	`MarketplaceDataFixRunner`	Preenche logo/capa faltantes das distribuidoras

Dica: Correção de 21/10/2026: o `data.sql` inseria produtos sem `codigo` (campo obrigatório) e travava a inicialização em banco vazio. Agora cada produto já vai com o código oficial do catálogo. Testado subindo o backend num banco vazio.

4.8 Tratamento de erros

Regras de negócio lançam `CadastroException(HttpStatus, mensagem)`. O `GlobalExceptionHandler` (@RestControllerAdvice) converte em JSON:

Exceção	Status	Mensagem
`CadastroException`	o informado (400/404/409/502)	a da exceção
`MethodArgumentNotValidException`	400	mensagens de validação dos campos (@NotBlank, @Size…)
`MissingServletRequestPart/Parameter`	400	“Arquivo de imagem é obrigatório.”
`HttpRequestMethodNotSupported`	405	“Operação não disponível. Reinicie o backend…”
`HttpMessageNotReadable`	400	“Dados inválidos. Verifique os campos informados.”

No app, `extractErrorMessage()` (`src/services/http.ts`) lê o campo `erro` e mostra ao usuário; se o servidor não estiver acessível, mostra “Não foi possível conectar ao servidor. Verifique se o backend está rodando.”

5. Banco de dados oficial (PostgreSQL)

Em 21/10/2026 o banco local foi **substituído pelo banco oficial do projeto**, montado pela equipe a partir das entidades do backend. Ele é uma base **limpa**: catálogo das distribuidoras parceiras e conta de demonstração, sem pedidos de teste. Os scripts, o dump e o diagrama passaram a fazer parte do repositório do backend, na pasta `database/`.

5.1 Conexão

Item	Valor
Servidor	`localhost:5432` · PostgreSQL 18 no Windows (serviço postgresql-x64-18)
Banco	`stock4event` (UTF-8, cerca de 9 MB)
Usuário / senha	`postgres` / a do `application.properties` (local: `admin`)
URL JDBC	`jdbc:postgresql://localhost:5432/stock4event`
Clientes gráficos	pgAdmin ou DBeaver (no DBeaver: botão direito no banco → View Diagram)

5.2 Arquivos da pasta database/

Arquivo	Para que serve
`01_schema.sql`	Cria as 19 tabelas com chaves primárias, 28 chaves estrangeiras e restrições únicas
`02_seed_catalogo.sql`	4 distribuidoras e 40 produtos (conteúdo do `data.sql`, com um gatilho temporário que preenche `codigo`)
`03_seed_demo.sql`	Empresa e usuário demo, endereço, cartão, PIX e formas de pagamento (o mesmo que os runners Java fazem)
`stock4event_full.sql`	Dump completo (estrutura + dados) do banco oficial
`build_from_scratch.sh`	Recria o banco do zero rodando 01, 02 e 03 — **apaga** o banco de destino
`restore_stock4event.sh`	Restaura um dump num banco (pede confirmação se ele já existir)
`dump_stock4event.sh`	Gera backup em `backups/` (SQL completo, só estrutura e formato compacto)
`gerar_er.py`	Regera o diagrama ER lendo o banco real (precisa do Graphviz)
`stock4event_er.png / .svg / .dot`	Diagrama ER (PNG para ver, SVG para ampliar, DOT é o código-fonte)
`GUIA_DE_USO.md`	Guia passo a passo: ligar, conectar, consultar, backup, restauração e cuidados

Nota: A pasta `database/backups/` fica fora do git (`.gitignore`), porque backups podem conter dados reais de contas.

Dica: Os scripts `.sql` começam com `SET client_encoding = 'UTF8';` (versão 4). Sem isso, o `psql` do Windows lia os arquivos como WIN1252: o script parava com “0x81 não tem equivalente” ou gravava acentos corrompidos.

5.3 Como montar o banco

Caminho	Comandos	Quando usar
A — pelo backend	`CREATE DATABASE stock4event;` e subir o backend	Mais simples: Hibernate cria as tabelas, `data.sql` e runners populam

Caminho	Comandos	Quando usar
B — script do zero	`./build_from_scratch.sh`	Montar sem Java, ou num banco de teste (`./build_from_scratch.sh stock4event_teste`)
C — restaurar o dump	`./restore_stock4event.sh stock4event_full.sql`	Voltar exatamente ao banco oficial

Os scripts `.sh` rodam no Git Bash e leem a senha de `PGPASSWORD`. No Windows, coloque `C:\Program Files\PostgreSQL\18\bin` no PATH antes.

5.4 Diagrama entidade-relacionamento

O diagrama abaixo foi gerado do banco real. As cores agrupam os domínios: **azul** contas (empresas, usuários, perfis), **verde** catálogo e quiosques, **laranja** pedidos e pagamentos, **lilás** logística, **amarelo** dados salvos da empresa e **roxo-claro** comunicação (e-mail de boas-vindas). Linha cheia = chave estrangeira obrigatória; tracejada = opcional. O arquivo `stock4event_er.svg` permite ampliar sem perder qualidade.

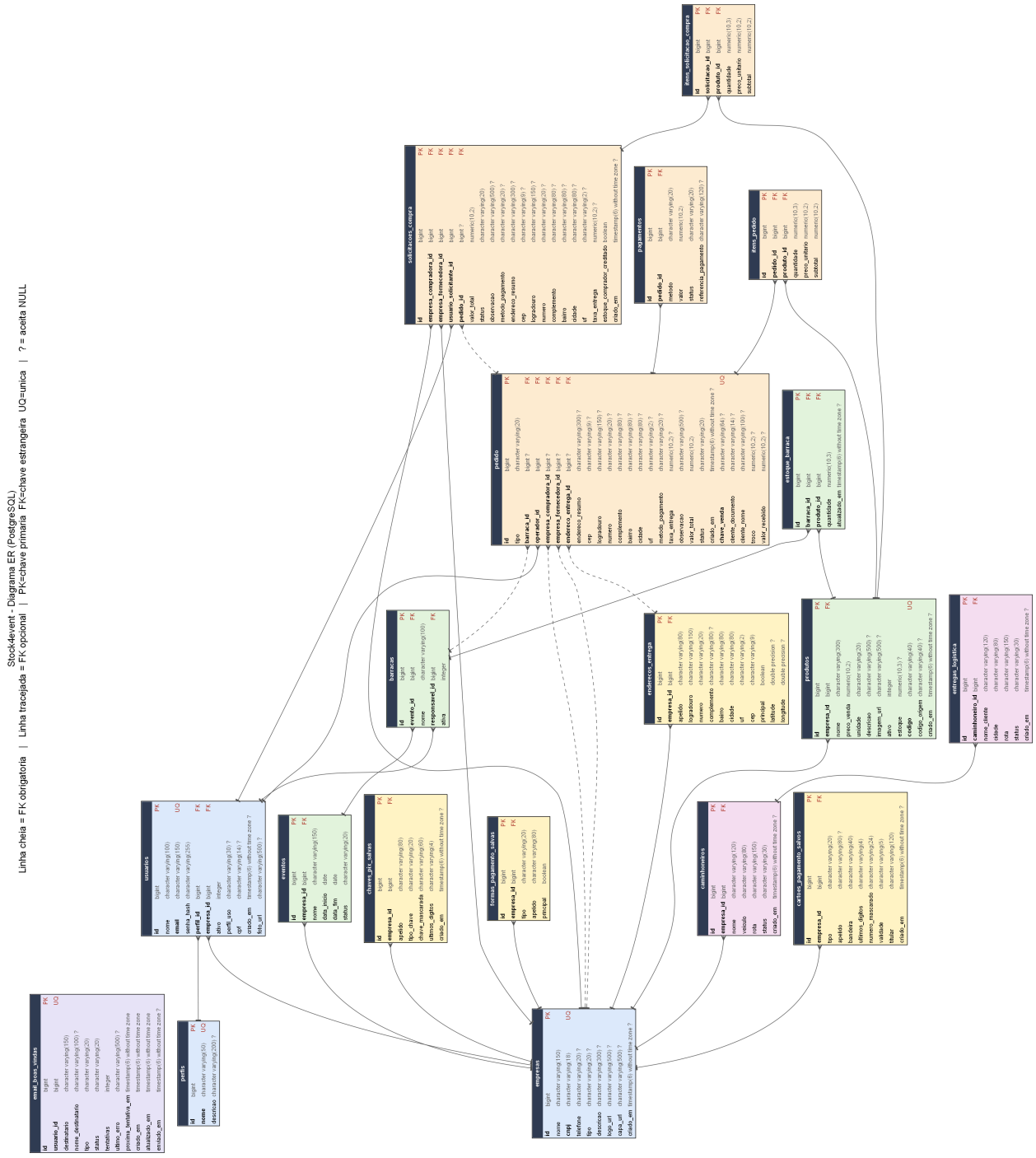


Figura 5 — Diagrama ER do banco stock4event (página girada para caber).

5.5 Tabelas e domínios

Domínio	Tabelas	Ligações principais
Contas	`perfis, empresas, usuarios`	usuário → empresa e perfil (obrigatórias)
Catálogo e quiosques	`produtos, eventos, barracas, estoque_barraca`	produto → empresa; barraca → evento e responsável; estoque_barraca liga barraca × produto
Pedidos e pagamentos	`solicitacoes_compra, itens_solicitacao_compra, pedido, itens_pedido, pagamentos`	solicitação → compradora, fornecedora, usuário e pedido; item → produto; pagamento → pedido

Domínio	Tabelas	Ligações principais
Logística	`caminhoneiros`, `entregas_logistica`	caminhoneiro → empresa; entrega → caminhoneiro
Dados salvos	`enderecos_entrega`, `formas_pagamento_salvas`, `cartoes_pagamento_salvos`, `chaves_pix_salvas`	todos → empresa
Comunicação	`email_boas_vindas`	usuario_id único; ligação lógica com usuarios, sem chave estrangeira

5.6 Restrições importantes

Restrição	Efeito
`empresas.cnpj` UNIQUE	Não existem duas empresas com o mesmo CNPJ/CPF (o cadastro devolve 409)
`usuarios.email` UNIQUE	Um e-mail por conta
`email_boas_vindas.usuario_id` UNIQUE	No máximo um e-mail de boas-vindas por conta, mesmo com novas tentativas
`perfis.nome` UNIQUE	Perfil “Admin” é único
`produtos.codigo` NOT NULL UNIQUE	Todo produto tem um código único (`MKT-…` no catálogo, `EMP-{empresa}-NNNN` nos demais)
`estoque_barraca (barraca_id, produto_id)` UNIQUE	Uma linha de estoque por produto em cada quiosque
`solicitacoes_compra.estoque_comprador_creditado` NOT NULL	Garante que o estoque do comprador é creditado uma única vez
28 chaves estrangeiras	Apagar um registro “pai” com “filhos” dá erro 23503 — a proteção funcionando

Em relação ao banco antigo, o oficial é **mais rígido**: `produtos.codigo` e `estoque_comprador_creditado` passaram a ser obrigatórios e há índices únicos explícitos. As entidades Java já exigiam esses campos, então o backend funciona sem alteração.

5.7 Dados iniciais do banco oficial

Tabela	Linhas	Conteúdo
`perfis`	1	Admin
`empresas`	4	Casa dos Vinhos, Cervejaria Caruaru, Whisky Labs (DISTRIBUIDOR) e Mercado Demo (CLIENTE)
`usuarios`	1	Conta Demo — demo@stock4event.com / demo1234, perfil de uso ClienteFornecedor
`produtos`	15	5 por distribuidora, com foto e estoque
`enderecos_entrega`	1	Loja — Av. Agamenon Magalhães, 100, Caruaru/PE (principal)
`formas_pagamento_salvas`	4	PIX (principal), crédito, débito e dinheiro
`cartoes_pagamento_salvos` / `chaves_pix_salvas`	1 / 1	Visa final 4242 · chave CNPJ mascarada
Demais tabelas	0	Sem pedidos, quiosques nem motoristas: base limpa para a demonstração

Catálogo oficial

Código	Produto	Preço (R$)	Estoque
MKT-CDV-001	Vinho Mazzei Zisola Sicilia Noto Rosso 750ml	156,00	150
MKT-CDV-002	Vinho Louis Latour Bourgogne Chardonnay 750 ml	200,00	100
MKT-CDV-003	Vinho Naturelle Tinto Reserva 750 ML	60,00	300
MKT-CDV-004	Vinho Carmin De Peumo Carmenere 750 ml	666,00	80
MKT-CDV-005	Vinho Carolina Reserva Sauvignon Blanc 750 ml	60,00	200
MKT-CC-001	Cerveja Heineken Long Neck 330ml	5,99	200
MKT-CC-002	Cerveja Corona Long Neck	6,99	150
MKT-CC-003	Cerveja Amstel Lata 269ml	3,19	300
MKT-CC-004	Cerveja Cerpa Export Long Neck 350ml	8,39	100
MKT-CC-005	Cerveja Paulaner Munchen Weissbier 500ml	11,19	80
MKT-WL-001	Whisky Escocês Royal Salute 21 Anos 700ml	989,00	150
MKT-WL-002	Whisky The Balvenie PortWood 21 Anos 700ml	2.795,30	100
MKT-WL-003	Whisky Glenfiddich Single Malt 12 Anos 750ml	415,75	300
MKT-WL-004	Conhaque Martell L'Or de Jean Martell 700ml	18.711,00	80
MKT-WL-005	Whisky Suntory Hibiki Japanese Harmony 700ml	729,90	200

CDV = Casa dos Vinhos · CC = Cervejaria Caruaru · WL = Whisky Labs. O dump guarda códigos provisórios `LEG-<id>`; o `ProdutoCatalogoFixRunner` os troca pelos códigos acima na primeira vez que o backend sobe.

5.8 Consultas úteis

```
-- Quantas linhas em cada tabela
SELECT relname AS tabela, n_live_tup AS linhas FROM pg_stat_user_tables ORDER BY relname;

-- Produtos de cada distribuidora
SELECT e.nome AS empresa, p.codigo, p.nome, p.preco_venda, p.estoque
FROM produtos p JOIN empresas e ON e.id = p.empresa_id
WHERE p.ativo = 1 ORDER BY e.nome, p.codigo;

-- Compras de uma empresa com o status
SELECT s.id, f.nome AS fornecedor, s.status, s.valor_total, s.criado_em
FROM solicitacoes_compra s JOIN empresas f ON f.id = s.empresa_fornecedora_id
WHERE s.empresa_compradora_id = 4 ORDER BY s.criado_em DESC;
```

5.9 Backup, restauração e a troca de 21/10/2026

- **Regra de ouro:** faça `./dump_stock4event.sh` antes de qualquer mudança de estrutura ou de dados.
- Teste comandos perigosos dentro de uma transação: `BEGIN; …; ROLLBACK;` Nunca rode DELETE/DROP sem WHERE num banco que importa.
- **Banco anterior:** antes da troca, o banco local tinha 33 empresas, 19 usuários (contas de teste da equipe), 110 solicitações de compra e 127 produtos. Ele foi salvo em `Área de Trabalho\Stock4event\BANCO DE DADOS\backups\banco_antigo_antes_da_troca_20261021_221936` (.sql e .dump); a restauração foi testada e devolveu todos os registros.
- **Validação da troca:** backend subindo em banco vazio e no banco oficial; login, sessão, marketplace, produtos, endereços, pagamentos, notificações, financeiro, quiosques e pedidos respondendo 200.

6. Identidade visual

Em 21/10/2026 o app trocou o logo em texto (“Stock” + “4event”) pela **identidade visual oficial**. A marca combina uma **caixa aberta** (o estoque), o número **4** e uma **seta âmbar para cima** (movimento e crescimento). Slogan oficial: **“Compras e estoque para eventos”**.

6.1 Arquivos originais da marca



Logo horizontal com slogan — fundos claros, documentos e apresentações



Avatar circular — redes sociais e abertura do app



Ícone — ícone do app, favicon e splash

6.2 Paleta de cores

Azul-marinho
#092D5E
fundo do ícone, telas do fornecedor e de entrada

Azul-escuro
#011C46
texto do logo, títulos

Âmbar
#FFAD19
“4event”, seta, telas do cliente, botões

Branco
#FFFFFF
logo sobre azul, cards

Azul-claro
#EAF1F8
fundos suaves do fornecedor

Antes	Depois	Onde mudou
#F8B125 (amarelo)	#FFAD19 (âmbar do logo)	Tema, telas do cliente, botões, indicadores — 83 usos
#123B6D	#092D5E	COMPANY_COLORS.primary (fornecedor)
#0E2A5C	#011C46 / #092D5E	Tom escuro do fornecedor e fundo das telas de entrada (AUTH_NAVY)

6.3 Versões do logo e quando usar

Versão (BrandLogo)	Arquivo	Uso
	`padrao` `logo.png`	Fundos claros
	`branco` `logo-branco.png`	Fundos azuis: cabeçalhos do fornecedor e telas de entrada

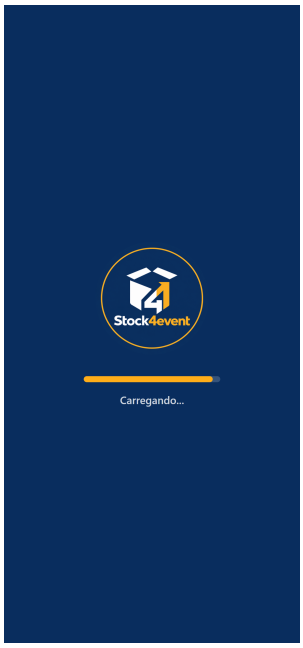
Versão (BrandLogo)	Arquivo	Uso
	`azul` `logo-azul.png`	Fundos âmbar: cabeçalhos do cliente (o “4event” âmbar sumiria no âmbar)
	`sloganBranco` `logo-horizontal-branco.png`	Abertura do app, com o slogan

6.4 Como a marca entrou no código

Onde	O que mudou
`src/components/brand/BrandLogo.tsx`	Componente novo que desenha o logo oficial. Props: `variant` (padrao, branco, azul, slogan, sloganBranco) e `height`
`src/assets/brand/`	Versões do logo geradas a partir dos arquivos originais + `avatar.png` e `marca-branca.png`
`BrandMark` · `AuthBrand`	Cabeçalhos do fornecedor e telas de entrada passaram a usar o BrandLogo; o AuthBrand ganhou a opção `slogan`
13 telas do cliente	O texto “Stock4event” do cabeçalho virou `<BrandLogo variant="azul" />`
`AnimationScreen`	“Carregando…” mostra o avatar circular; depois o logo com o slogan oficial (saiu “Agilidade que conecta negócios”)
`theme.ts` · `authTheme.ts`	Nova paleta, documentada no topo do arquivo
`app.json` + `assets/`	Ícone, favicon, splash e ícone adaptativo do Android com a caixa da marca; fundo da splash em #092D5E

Atenção: No celular, o novo ícone e a nova splash só aparecem depois de um novo build do app. No Expo Go continua o ícone do Expo. Commit da identidade: `42a4263` (branch leticia), já mesclado no main pelo PR #22.

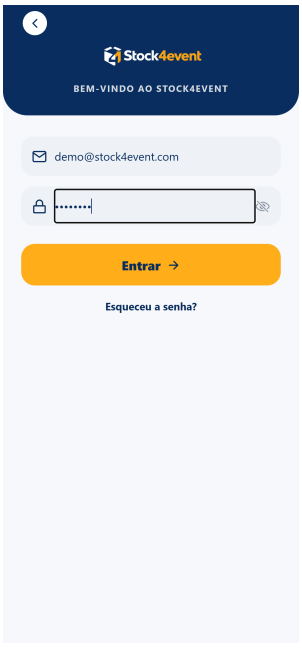
6.5 Telas com a nova identidade



Abertura com o avatar



Boas-vindas



Login



Escolha de perfil



Fornecedor — início



Fornecedor — produtos



Fornecedor — vendas



Fornecedor — gráficos



Cliente — início



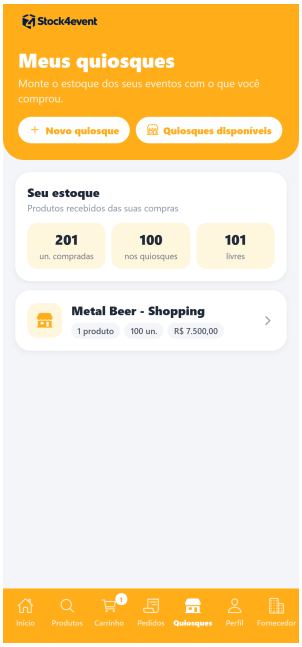
Vitrine da distribuidora



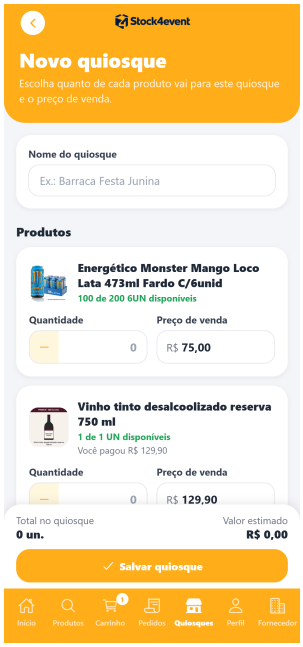
Carrinho



Acompanhamento do pedido



Meus quiosques



Novo quiosque



Perfil



Pagamento com PIX

As 45 capturas completas estão em `Área de Trabalho\Stock4event\CAPTURAS DE TELA DO PROJETO`. A apresentação padronizada (16 slides) usa a mesma marca: `SLIDE\Stock4event_Apresentacao_Padronizada.pptx`.

6.6 Mensagens da marca

Uso	Texto
Slogan	Compras e estoque para eventos
Frase de abertura	Do pedido ao quiosque.
Assinatura	Estoque conectado. Negócio em movimento.
Chamada em três verbos	Compre · Acompanhe · Revenda

7. Frontend (aplicativo mobile)

7.1 Estrutura de pastas

```
Mobile/
├─ App.tsx              raiz: providers + AppNavigator
├─ index.ts             registra o App no Expo
├─ app.json             nome, ícones, splash (Expo)
├─ .env.local           login pré-preenchido (fora do git)
└─ src/
   ├─ config/           api.ts (URL do backend), demo.ts (conta demo)
   ├─ services/         chamadas HTTP e AsyncStorage (16 arquivos)
   ├─ context/          estado global (5 contexts)
   ├─ navigation/       AppNavigator, AuthNavigator, MainNavigator, types.ts
   ├─ screens/          telas: Tela.tsx + useTela.ts (lógica) + styles.ts
   ├─ components/       cabeçalhos, barras inferiores, modais, botões, inputs
   ├─ hooks/            useAppGoBack
   ├─ theme/            cores por perfil, espaçamentos, fontes
   ├─ types/            tipos compartilhados (auth, produto, pedido…)
   └─ utils/            funções puras (estoque, CEP, cartão, PIX, datas) + testes
```

Padrão das telas: cada tela tem três arquivos — o componente visual (`Tela.tsx`), um hook com a lógica e as chamadas à API (`useTela.ts`) e os estilos (`styles.ts`). Isso separa o "como aparece" do "o que faz".

7.2 Inicialização do app

O `App.tsx` monta a árvore de *providers*. A ordem importa: quem está por fora fica disponível para quem está por dentro.

```
<GestureHandlerRootView>
  <SafeAreaProvider>
    <ConfirmDialogProvider>          diálogos de confirmação globais
      <AuthProvider>                 sessão, usuário, perfil
        <AppNavigator>
          isLoading?      → spinner
          autenticado?    → <PurchaseCartProvider><MainNavigator/></PurchaseCartProvider>
          não autenticado → <AuthNavigator/>
```

Os providers de produtos e quiosques (`ProductsProvider`, `QuiosqueProvider`) são montados dentro do `MainNavigator`.

7.3 Conexão com o backend

Toda comunicação usa a **Fetch API** nativa (não há axios). Três arquivos fazem a ponte:

Arquivo	Função
`src/config/api.ts`	`API_BASE_URL = http://{host}:8081`, com o host resolvido automaticamente; `getImageUrl(path)` monta URLs de imagens
`src/services/http.ts`	`apiFetch()`, `parseResponse<T>()` (lança Error com a mensagem do backend se `!response.ok`) e `extractErrorMessage()`
`src/services/*Service.ts`	Uma função por endpoint, tipada; normalizam números que vêm como string (ex.: `precoVenda`, `estoque`)

Service	Endpoints que consome	Usado por
`authService`	/api/cadastro, /api/usuarios/login, /me, /recuperar-senha, PUT /usuarios/{id}, PUT /empresas/{id}, /usuarios/upload, /empresas/upload	Login, Register, ForgotPassword, subtelas de Ajustes, AuthContext
`productService`	/api/produtos (CRUD + upload)	ManageProducts, Home, ProductsContext, quiosques
`marketplaceService`	/api/marketplace/*, /api/solicitacoes-compra/*	Explorar, Cart, StoreVitrine, ProductDetail, Sacola, Pedidos, Acompanhamento, Vendas
`barracaService`	/api/barracas	Quiosque (fornecedor), ClienteQuiosques, QuiosqueContext
`enderecoService`	/api/enderecos (+ /cep/{cep})	Endereços, Sacola
`formaPagamentoService`	/api/formas-pagamento	FormasPagamento, Sacola
`cartaoPagamentoService`	/api/cartoes-pagamento	FormasPagamento, modais de cartão
`pixChaveService`	/api/chaves-pix (migra chaves antigas salvas localmente)	FormasPagamento
`financeiroService`	/api/financeiro/resumo, /stock-dia	Home (fornecedor), Cards
`notificacaoService`	/api/notificacoes	NotificationsModal (sino)
`logisticaService`	/api/caminhoneiros, /api/logistica	*Nenhuma tela ainda* (arquivo novo, não versionado)
`sessionStorage` / `perfilStorage` / `purchaseCartStorage`	AsyncStorage (local)	AuthContext, PurchaseCartContext

7.4 Autenticação e sessão no app

O `AuthContext` é o coração do app. Ele expõe `user`, `token`, `isAuthenticated`, `perfilUso`, `perfilConta`, `podeAlternarPerfil` e as ações `signIn`, `signOut`, `updateUser`, `setPerfilUso`, `alternarPerfil`.

- **signIn:** chama o login, calcula `expiresAt = agora + expiresIn`, salva em `@stock4event_session` e aplica a sessão.
- **Ao abrir o app:** `restoreSession()` lê a sessão salva; se expirou, apaga; senão chama `/api/usuarios/me` para validar o token e atualizar o usuário. Se falhar, desloga.
- **Perfil:** o perfil salvo no aparelho no cadastro tem prioridade; senão usa o `perfilUso` vindo do servidor. Contas “ClienteFornecedor” ficam na visão Fornecedor — só a conta demo pode alternar (`isContaDemo()`).
- **Tokens legados:** se o backend antigo devolver só o usuário, o app cria um token `legacy:{id}` para continuar funcionando.

7.5 Navegação

Usa `@react-navigation/native-stack` com uma pilha única por estado (deslogado/logado). As barras inferiores são componentes próprios (`BottomTabBar` e `ClienteTabBar`), não o pacote de tabs, para ter uma barra diferente por perfil, com a cor do perfil e badge no carrinho.

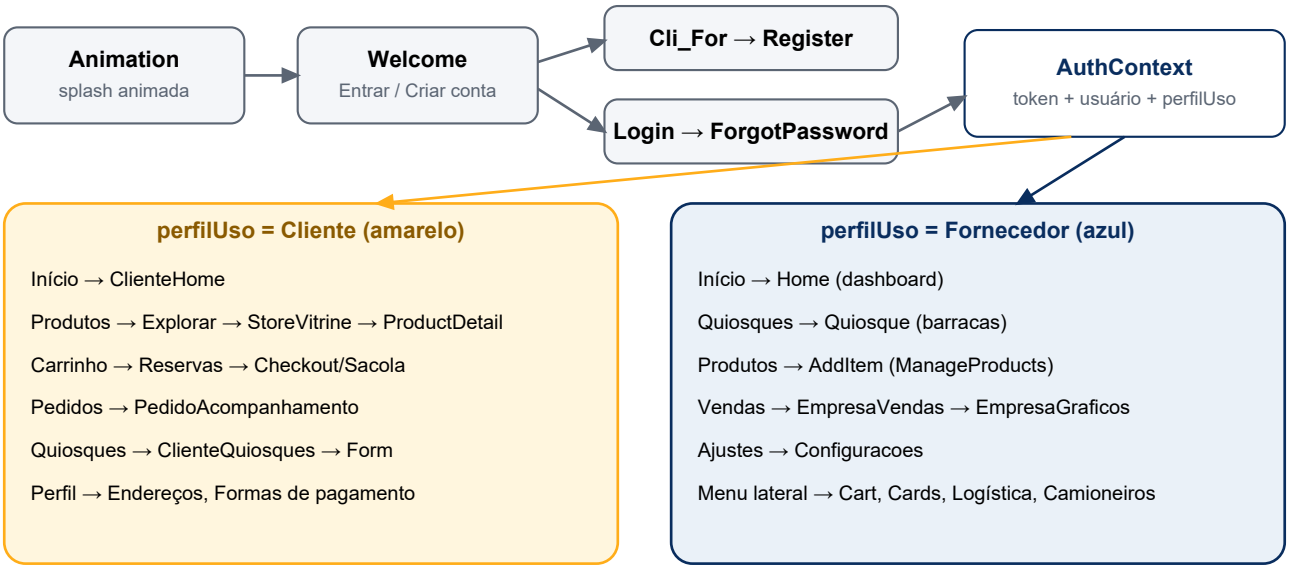


Figura 3 — Fluxo de navegação e telas por perfil.

7.6 Contextos (estado global)

Context	Guarda	Origem dos dados
`AuthContext`	Usuário logado, token, perfil ativo	API + AsyncStorage
`ProductsContext`	Produtos da empresa logada + `refresh`	`GET /api/produtos?empresaId=`
`QuiosqueContext`	Quiosques da empresa logada	`GET /api/barracas?empresaId=`
`PurchaseCartContext`	Sacola de compras agrupada por fornecedor; taxa de entrega R$ 7,00 por fornecedor	AsyncStorage `@stock4event_purchase_cart_{empresaId}`
`ConfirmDialogContext`	Diálogo de confirmação reutilizável (ex.: “Remover produto?”)	Local

7.7 Telas

Tela (rota)	Perfil	O que faz	Backend
Animation, Welcome	Todos	Splash animada e tela inicial com Entrar/Criar conta	—
Cli_For → Register	Todos	Escolha Cliente/Fornecedor e cadastro de empresa + usuário; confere o formato do e-mail	POST /cadastro
Login, ForgotPassword	Todos	Entrar; redefinir senha por e-mail	/usuarios/login, /recuperar-senha
Home	Ambos	Fornecedor: dashboard (produtos, financeiro). Cliente: renderiza ClienteHome (destaques de fornecedores)	/produtos, /financeiro, /marketplace
Explorar	Cliente	Busca de fornecedores e produtos	/marketplace
StoreVitrine	Ambos	Vitrine de um fornecedor (capa, logo, grade de produtos)	/marketplace/fornecedores/{id}/produtos
ProductDetail	Ambos	Detalhe do produto, quantidade e “Adicionar à sacola”	—
Reservas (Carrinho), Sacola, Checkout	Ambos	Revisa itens por fornecedor, escolhe endereço e pagamento, finaliza	POST /solicitacoes-compra
Pedidos	Cliente	Histórico de compras	GET /solicitacoes-compra

Tela (rota)	Perfil	O que faz	Backend
PedidoAcompanhamento	Ambos	Linha do tempo, previsão, cancelar; polling a cada 5 s. Fornecedor vê pedidos recebidos e muda status	GET /{id}, PATCH /status, /cancelar
ClienteQuiosques / ClienteQuiosqueForm	Cliente	Monta quiosques com o estoque comprado, define preço de venda	/barracas, PUT /produtos/{id}
Quiosque	Fornecedor	Quiosques/filiais com alocação de produtos (BarracaFormModal)	/barracas
AddItem (ManageProducts)	Fornecedor	CRUD de produtos com foto (ProductFormModal)	/produtos, /produtos/upload
EmpresaVendas, EmpresaGraficos	Fornecedor	Pedidos recebidos, resumo de vendas e gráficos	/solicitacoes-compra/recebidas
Cart	Fornecedor	Marketplace do fornecedor (comprar de outras distribuidoras) + últimos pedidos	/marketplace, /solicitacoes-compra
Cards	Fornecedor	Carteira (dados fictícios) e Estatísticas (API)	/financeiro
Logistica, Camioneiros, CadastroCamioneiros	Fornecedor	Motoristas e status de entrega	*dados fixos na tela*
Configuracoes (Ajustes)	Ambos	Menu: alternar visão (demo) e itens que abrem as subtelas abaixo	—
PerfilLoja	Fornecedor	Foto, capa (recorte 16:9) e descrição da loja	/empresas/upload, PUT /empresas
DadosPessoais	Ambos	Nome e e-mail; no cliente, também a foto de perfil	/usuarios/upload, PUT /usuarios
AlterarSenha	Ambos	Nova senha com confirmação	PUT /usuarios
DadosEmpresa	Ambos	Nome, CNPJ e telefone (no cliente, “Dados do negócio”, sem telefone)	PUT /empresas
Perfil	Cliente	Dados da conta com a foto de perfil, atalhos para endereços e pagamentos, sair	—
Enderecos, FormasPagamento	Ambos	CRUD de endereços (com busca de CEP), formas, cartões e PIX	/enderecos, /formas-pagamento, /cartoes-pagamento, /chaves-pix

7.8 Componentes reutilizáveis

Componente	Uso
`ScreenHeader / BackTitleHeader`	Cabeçalhos do fornecedor: abas principais (menu lateral + sino) e telas internas (botão voltar)
`TabScreenLayout / PageContent`	Esqueleto padrão das telas com cabeçalho, título e subtítulo
`BottomTabBar / ClienteTabBar`	Barras inferiores por perfil; incluem o botão de troca de visão da conta demo (`useTrocarVisaoDemo`)
`HamburgerButton`	Menu lateral do fornecedor feito com Modal (70% da largura)
`HeaderActions / HeaderCartBadge / CalendarModal`	Ações do cabeçalho, badge com itens da sacola, seletor de data
`NotificationsModal`	Lista de notificações; marca como lidas localmente; navega para pedido ou vitrine

Componente	Uso
`CheckoutPaymentModal`, `CartaoFormModal`, `PixFormModal`, `EnderecoFormModal`	Modais de pagamento, cartão, PIX e endereço
`RemoteImage + imageFallback`	Imagem da API com fallback para avatar de iniciais quando falha
`CustomButton`, `PagePrimaryButton`, `CustomInput`	Botões em pílula e campos de formulário padronizados
`AuthScreenShell`, `AuthBrand`, `BottlesHero`	Visual das telas de login/cadastro

7.9 Regras no app e testes

As regras dos quiosques do cliente ficam em `src/utils/estoqueQuiosque.ts` como **funções puras** e têm **22 testes** em `src/utils/__tests__/estoqueQuiosque.test.ts`:

- `disponivelParaQuiosque()`: disponível = estoque comprado − quantidade já colocada nos *outros* quiosques.
- `resumoEstoque()`: totais comprado / nos quiosques / livre.
- `validarQuiosqueForm()`: nome obrigatório, ao menos um produto, quantidade ≤ disponível, preço > 0 quando há quantidade.
- `precoPagoPorNome()`: mostra “Você pagou R$ X” buscando o preço no pedido entregue mais recente com item de mesmo nome.
- Outros utilitários: `cartaoUtils` (bandeira, máscara, validade, CVV, Luhn), `pixUtils` (máscaras e validação de chaves), `cepUtils`, `dateFormat`.

7.10 Identidade visual no código

As cores ficam em `src/theme/theme.ts` (`COMPANY_COLORS` para o fornecedor, `CLIENTE_COLORS` para o cliente) e em `src/theme/authTheme.ts`. O logo é sempre desenhado pelo componente `BrandLogo`. Regras e arquivos da marca no **capítulo 6**.

7.11 Dados salvos no aparelho (AsyncStorage)

Chave	Conteúdo
`@stock4event_session`	Token, usuário e data de expiração
`@stock4event_perfil_uso:{email}`	Perfil escolhido no cadastro
`@stock4event_modo_ativo:{email}`	Visão ativa da conta demo (Cliente/Fornecedor)
`@stock4event_purchase_cart_{empresaId}`	Sacola de compras
(notificacoesStorage)	IDs de notificações já lidas, por empresa
`@quickstock_pix_chaves_{empresaId}`	Chave legada: chaves PIX antigas, migradas para o backend e apagadas (mantida de propósito)

8. Fluxos de ponta a ponta

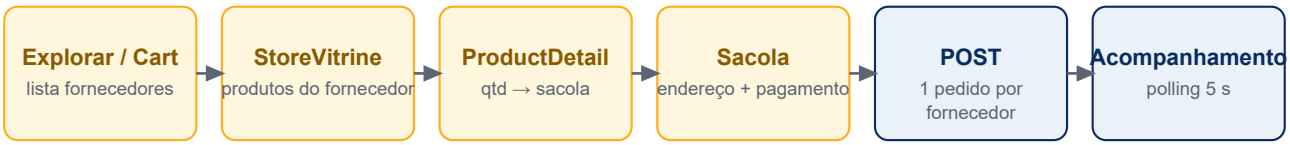
8.1 Cadastro e login

```
Welcome → "Criar conta" → Cli_For (Cliente | Fornecedor) → Register
    └ POST /api/cadastro  {empresa, usuario, perfil}
        └ CadastroService: cria empresa (tipo pelo perfil) + usuário Admin (BCrypt)
            + registro em email_boas_vindas (mesma transação)
        └ após o commit, em segundo plano: e-mail de boas-vindas (cliente ou fornecedor)
    └ volta para Welcome com mensagem de sucesso

Welcome → "Entrar" → Login
    └ POST /api/usuarios/login → {token, expiresIn, usuario}
        └ AuthContext salva @stock4event_session → MainNavigator

Reabrir o app → loadSession() → GET /api/usuarios/me (Bearer) → restaura ou desloga
```

8.2 Compra no marketplace



PurchaseCartContext guarda a sacola (AsyncStorage por empresa) até o checkout

Figura 4 — Jornada de compra.

- A sacola pode ter produtos de **vários fornecedores**. No checkout, o app faz **um POST por fornecedor** (cada um vira uma solicitação separada, com R$ 7,00 de entrega).
- Se todos derem certo, limpa a sacola, recarrega os produtos e abre o acompanhamento do primeiro pedido (com a lista de todos os IDs). Se algum falhar, mostra quais fornecedores falharam.
- No acompanhamento, o app consulta `GET /api/solicitacoes-compra/{id}` a cada 5 s; o backend avança o status (20 s → em rota, 55 s → entregue) e, ao entregar, credita o estoque do comprador.
- Enquanto está “aguardando liberação”, o comprador pode cancelar (o estoque volta ao fornecedor e o pagamento é estornado).

8.3 Fornecedor gerencia pedidos recebidos

```
EmpresaVendas → GET /api/solicitacoes-compra/recebidas?empresaFornecedoraId=X
  └ PATCH /api/solicitacoes-compra/{id}/status {empresaFornecedoraId, status}
       aguardando_liberacao → em_rota → entregue   (ou → cancelada)
```

8.4 Cliente monta quiosques com o que comprou

```
Pedido entregue → backend cria/atualiza produtos na empresa do cliente (codigo_origem)
ClienteQuiosques → resumo: comprado · nos quiosques · livre
  └ ClienteQuiosqueForm: por produto "N de M disponíveis", "Você pagou R$ X",
    quantidade + preço de venda
      ├ PUT /api/produtos/{id}      (novo preço de venda)
      └ POST|PUT /api/barracas      (empresaId do cliente, itens)
```

8.5 Cadastro de produto com foto

```
AddItem → ProductFormModal
  ├─ expo-image-picker escolhe a foto → expo-image-manipulator reduz
  ├─ POST /api/produtos/upload (multipart "file") → {imagemUrl}
  └─ POST /api/produtos {nome, precoVenda, unidade, descricao, imagemUrl, estoque, empresaId}
       └─ backend gera o código EMP-{empresaId}-NNNN
```

9. Estado atual, limitações e pontos de atenção

9.1 O que está integrado de verdade

- Cadastro, login, sessão JWT e recuperação de senha.
- CRUD de produtos com upload de imagem.
- Quiosques do fornecedor e do cliente (com estoque).
- Marketplace completo: vitrine, sacola multi-fornecedor, checkout, acompanhamento, cancelamento.
- Fornecedor alterando status dos pedidos recebidos.
- Endereços (com ViaCEP), formas de pagamento, cartões e chaves PIX salvos no backend.
- Notificações (geradas pelo backend) e estatísticas financeiras.

9.2 O que ainda é demonstração/mock

Item	Situação
Pagamento	Simulado: não há gateway; PIX/cartão são aprovados na hora
Status automático	Pedidos avançam por tempo (20 s / 55 s) só para a apresentação
Home do fornecedor	Parte dos cards/gráfico com valores fixos na tela
Cards → Carteira	Dados fictícios
Logística / Caminhoneiros	Telas usam lista fixa; a API existe mas o `logisticaService.ts` ainda não está ligado a nenhuma tela
Notificações	Promoções/ofertas são textos fixos no código; “lida” só no aparelho

9.3 Situação dos repositórios

Repositório	Situação em 22/10/2026
Backend (`integracao-luan` → `leticia`)	Tudo o que este documento descreve está no GitHub (último commit da versão 4: `01371ae`). Fora do controle de versão ficam só arquivos locais: `CnpjConsultaService/CnpjConsultaDTO` (em desenvolvimento) e `package-lock.json`
Frontend (`leticia`)	Tudo no GitHub (último commit: `579d2bb`). Fora do controle de versão: `.idea/workspace.xml` e o PDF desta documentação

9.4 Riscos técnicos

Dica: Autorização nas rotas — resolvido na versão 3: as rotas exigem JWT e cada empresa só acessa os próprios dados (acesso a outra empresa → 403; sem token → 401).

Risco: Segredos no repositório: `jwt.secret` e senha do banco estão no `application.properties`. Em produção, usar variáveis de ambiente.

Risco: Recuperação de senha sem verificação: basta saber o e-mail para trocar a senha de alguém.

Atenção: E-mail sem confirmação de titularidade: o cadastro valida só o formato do endereço. O e-mail de boas-vindas não prova que o endereço pertence a quem se cadastrou e não marca nada como verificado.

Atenção: ddl-auto=update + data.sql sempre: ótimo para desenvolvimento, arriscado em produção (usar Flyway/Liquibase e `validate`).

Atenção: Controllers expondo entidades: `/api/empresas, /api/pedidos, /api/pagamentos` devolvem a entidade JPA direto (inclui relacionamentos e pode expor dados).

Atenção: Concorrência de estoque: no PDV o estoque é travado (`PESSIMISTIC_WRITE`) e a venda é idempotente. No checkout do marketplace o débito ainda não usa trava; duas compras simultâneas do último item podem passar.

9.5 Documentação existente que está desatualizada

- `Stock4Event-FrontEnd/docs/diagramas/*.png`: porta 8080 e banco “quickstock”; o ER oficial atualizado agora é `database/stock4event_er.png`.
- `APRESENTACAO_STOCK4EVENT.md`: diz que as chaves PIX ficam só no aparelho (hoje vão para o backend).
- O `.docx` e o PDF ABNT do frontend ainda citam o nome antigo do repositório (os geradores já foram corrigidos).
- `docs/scripts/gerar_documentacao.mjs`: se rodado, sobrescreve a DOCUMENTACAO_OFICIAL com um esboço — não rodar sem revisar.

9.6 Próximos passos (roadmap da apresentação)

Frente	O que fazer
Segurança	Validar CNPJ na Receita (o serviço já existe), refresh token e segredos em variáveis de ambiente
Pagamento real	Integrar PIX / Mercado Pago no lugar do pagamento simulado
Logística	Ligar motoristas e rotas à API e calcular a distância até as lojas (latitude/longitude já no banco)
Engajamento	Notificações push e histórico de leitura no servidor
Publicação	Docker, deploy em nuvem e publicação nas lojas de apps
Banco	Adotar Flyway a partir do `01_schema.sql` e trocar `ddl-auto=update` por `validate`

Modelo de negócio (proposta da apresentação)

Hipóteses ainda não validadas com clientes: **comissão por pedido** paga pela distribuidora; **destaque pago** para distribuidoras na vitrine e nas notificações; e **plano de gestão** por assinatura para quem opera muitos quiosques.

10. Novidades da versão 3 (22/10/2026)

Este capítulo reúne tudo o que foi feito depois da versão 2, mostrando como front-end, back-end e banco de dados ficaram ligados.

10.1 Segurança: JWT em todas as rotas

Peça	Onde	O que faz
Filtro JWT	`security/JwtAuthenticationFilter`	Lê o `Authorization: Bearer`, valida o token e identifica o usuário
Regras de rota	`security/SecurityConfig`	Sessão stateless; públicos só login, cadastro, recuperar-senha e `/uploads/**`
Empresa do token	`security/AcessoService`	Resolve a empresa logada; `empresaId` diferente na URL → 403
App	`src/services/http.ts`	`apiFetch` envia o token em toda chamada; resposta 401 faz logout automático

- **401** sem token ou token inválido/expirado; **403** ao tentar acessar dados de outra empresa.
- Campos internos (empresa, ids, status) não são aceitos do corpo da requisição.
- Testes de integração rodam no banco exclusivo `stock4event_test` — nunca no banco real.

10.2 PDV (ponto de venda)

Tela própria no app (menu → **PDV**) para vender no balcão de um quiosque.

Camada	Itens
Front-end	`src/screens/Pdv/` (busca por nome/código/código de barras, carrinho, totais, troco, cliente opcional, comprovante “sem valor fiscal” via expo-print)
API	`GET /api/pdv/produtos?barracaId&busca`, `POST /api/pdv/vendas` (201 nova / 200 repetida), `GET /api/pdv/vendas/{id}`
Serviço	`VendaPdvService`: tudo numa transação, trava o estoque do quiosque, bloqueia estoque negativo, calcula total e troco no servidor
Banco	Tabela `pedido` ganhou `chave_venda` (única, evita venda duplicada), `valor_recebido`, `troco`, `cliente_nome`, `cliente_documento`

> **Nota:** O pagamento do PDV fica com status **pendente**: o app não simula aprovação de cartão/PIX.

10.3 Novas distribuidoras e catálogos

Distribuidora	O que mudou	Códigos
SaborViva	Nova distribuidora (CNPJ 30.004.004/0001-04) com 15 produtos, preços, estoque e imagens do catálogo	`MKT-SV-001..015`
Casa dos Vinhos	Catálogo trocado pelo novo catálogo sem álcool; os 5 produtos antigos foram desativados	`MKT-CDV-006..020`

Os dados estão no `02_seed_catalogo.sql` e no `data.sql`; as imagens ficam em `static/uploads`.

10.4 Vitrine entre contas

Recurso	Front-end	Back-end / banco
Adicionados pelos fornecedores	Tela Produtos (destaques) + “Ver todos”	`GET /api/marketplace/produtos-recentes`; nova coluna `produtos.criado_em` (mais novos primeiro)
Distribuidoras Parceiras	Antes “Quiosques de fornecedores”; botão “Ver todas”	`GET /api/marketplace/fornecedores`
Distribuidoras disponíveis	Home do cliente (antes “Quiosques disponíveis”)	Mesma API de fornecedores
Quiosques disponíveis	Botão na tela de quiosques → `QuiosquesDisponiveisScreen`	`GET /api/marketplace/quiosques?busca`: quiosques ativos das outras contas, sem expor quantidades

- Todo produto cadastrado por uma conta Fornecedor ou Cliente e fornecedor (tipo AMBOS) aparece para todos os clientes.
- O próprio produto aparece marcado como “Seu produto” e não pode ser comprado pela mesma empresa.
- A conta demo agora é do tipo **AMBOS** (`03_seed_demo.sql` e ajuste automático no backend).

10.5 Correções

- **Cancelar pedido:** o botão usava `Alert.alert`, que não aparece no navegador; agora usa o diálogo de confirmação do app. Se o pedido já saiu para entrega, o erro aparece no diálogo e a tela recarrega o status.
- **Ver todos / Ver todas:** passaram a abrir a nova tela `ExplorarLista` com a lista completa.
- Serialização de proxies do Hibernate em eventos/pedidos corrigida (`findById` no lugar de `getReferenceById`).

10.6 Banco de dados atualizado

Arquivo (pasta BANCO DE DADOS e `database/`)	Situação
`01_schema.sql`	Inclui as colunas do PDV, `produtos.criado_em` e `enderecos_entrega.latitude/longitude`; tipos de empresa corrigidos
`02_seed_catalogo.sql`	Catálogos atuais: Cervejaria Caruaru, Whisky Labs, Casa dos Vinhos (novo) e SaborViva
`03_seed_demo.sql`	Conta demo como AMBOS (demo@stock4event.com / demo1234)
`stock4event_full.sql` + backups de 22/10	Dump do banco real atual; os dumps antigos foram para `backups/anteriores`
`stock4event_er.png / .svg / .dot`	Diagrama ER refeito a partir do banco atual (Figura 5), já com as colunas novas

Conferência feita: o banco montado do zero com `build_from_scratch.sh` tem exatamente as mesmas tabelas e colunas do banco real usado pelo backend.

10.7 Testes automatizados

Projeto	Comando	Resultado
Backend	`mvn test` (banco `stock4event_test`)	142 testes passando: autenticação, isolamento, validação, PDV, catálogo, vitrine
Frontend	`npx tsc --noEmit` e `npx jest`	Tipos OK; 48 testes passando

11. Novidades da versão 4 (22/10/2026)

Este capítulo reúne o que foi feito depois da versão 3: a tela de Ajustes reorganizada, a identidade visual das contas (perfil da loja e foto de perfil) e o e-mail de boas-vindas enviado após o cadastro, com o banco de dados atualizado.

11.1 Ajustes em subtelas

A tela Ajustes deixou de ser um formulário único e virou um **menu**: cada item abre uma subtela com o próprio botão de salvar, que envia ao backend só os campos daquela subtela. Assim, trocar a senha não exige revalidar o CNPJ, por exemplo.

Perfil	Itens do menu
Fornecedor	Perfil da loja · Dados pessoais · Alterar senha · Dados da empresa
Cliente	Dados pessoais (com foto) · Dados do negócio · Alterar senha

- Telas em `src/screens/Configuracoes/`: `PerfilLojaScreen, DadosPessoaisScreen, AlterarSenhaScreen, DadosEmpresaScreen`.
- Componentes `AjustesSubtela` (cabeçalho, voltar, card e salvar), `AjustesMenuItem` e `AjustesCampo`: mudam de visual pelo perfil ativo (azul do fornecedor; azul e âmbar do cliente), então a mesma subtela serve aos dois.
- O backend já aceitava atualização parcial: campos ausentes no `PUT` não são alterados.

11.2 Perfil da loja e foto de perfil

Camada	Perfil da loja (fornecedor)	Foto de perfil (cliente)
App	Capa (recorte 16:9), foto (1:1) e descrição de até 300 caracteres; imagens reduzidas no aparelho e enviadas ao salvar	Avatar em Dados pessoais (recorte 1:1); aparece em Meu perfil
API	`POST /api/empresas/upload` + `PUT /api/empresas/{id}` com `descricao, logoUrl, capaUrl`	`POST /api/usuarios/upload` + `PUT /api/usuarios/{id}` com `fotoUrl`
Banco	Colunas já existentes `empresas.descricao, logo_url, capa_url`	Coluna nova `usuarios.foto_url`
Onde aparece	Vitrine, Explorar e início do cliente (já usavam esses campos)	Cabeçalho de Meu perfil

> **Dica:** Por segurança, logo, capa e foto só aceitam imagens enviadas ao próprio servidor (`/uploads/...`); um link externo é ignorado e o valor atual é mantido. Texto vazio remove a imagem.

11.3 E-mail de boas-vindas

Depois do cadastro de uma conta nova, o backend envia um e-mail de boas-vindas ao endereço informado, na versão do perfil salvo. Textos e banners vieram do material “E-mail Alert” da equipe.

Perfil salvo	Versão	Assunto
`Cliente`	CLIENTE (banner amarelo)	Bem-vindo ao Stock4event! Seu evento começa aqui.

Perfil salvo	Versão	Assunto
`Fornecedor` e `ClienteFornecedor`	FORNECEDOR (banner azul)	Bem-vindo ao Stock4event! Vamos conectar seu negócio a novos eventos.



Versão cliente



Versão fornecedor

Figura — Banners do e-mail de boas-vindas, com o mascote Stockinho.

Como o envio funciona

```
POST /api/cadastro
  └ transação do cadastro: empresa + usuário + email_boas_vindas (PENDENTE)
      cadastro falhou?  → tudo é desfeito, nenhum e-mail
  └ após o commit, em segundo plano (@Async + AFTER_COMMIT): 1ª tentativa
      aceito pelo SMTP      → ENVIADO
      falha temporária      → PENDENTE de novo, espera 1, 2, 4 min... (até 5 tentativas)
      falha definitiva      → FALHOU (ex.: endereço inválido)
  └ BoasVindasReprocessador (a cada minuto): refaz pendentes vencidos
```

- **Sem duplicidade:** `usuario_id` é único e cada tentativa reserva o registro com um `UPDATE … WHERE status = 'PENDENTE'`; execuções simultâneas nunca enviam a mesma mensagem. Tentativa interrompida sem resposta vira `INDETERMINADO` e não é reenviada sozinha.
- **A conta nunca é desfeita** por falha no e-mail, e a resposta 201 do cadastro não espera o envio.
- **Layout:** HTML em tabelas com estilos inline (Gmail e Outlook), botão “Acessar minha conta” em HTML e VML, link de reserva, texto alternativo nas imagens e versão em texto simples. Banners incorporados por CID (cerca de 145 KB cada).
- **Segurança:** o nome do usuário é escapado no HTML; credenciais só em variáveis de ambiente; o log mostra o e-mail mascarado.

11.4 Configuração do envio

Variável de ambiente	Para que serve
`STOCK4EVENT_EMAIL_HABILITADO`	Liga o envio (padrão `false`: o cadastro funciona e nada é enviado)
`STOCK4EVENT_EMAIL_REMETENTE`	Remetente, verificado no provedor de e-mail
`STOCK4EVENT_APP_URL`	Endereço real do app, usado no botão (http/https)
`SPRING_MAIL_HOST / _PORT / _USERNAME / _PASSWORD`	Servidor SMTP e credenciais do provedor

> **Nota:** Com o envio ligado e alguma variável faltando, o backend não sobe e diz qual falta. Modelo sem segredos: `email.env.example`; guia completo (provedor, SPF/DKIM/DMARC, teste local com Mailpit e produção): `docs/EMAIL_BOAS_VINDAS.md`.

Atenção: **Pendente para envio real:** contratar/configurar um provedor SMTP, verificar o remetente ou o domínio e definir a URL publicada do app.

11.5 Validação e confirmação de e-mail

- O cadastro passou a exigir domínio com ponto (`ana@exemplo` é recusado), no backend (`UsuarioRequestDTO`) e no app (`RegisterScreen`).
- Formato válido não prova que o endereço existe nem que pertence a quem se cadastrou.
- O projeto ainda não tem confirmação de e-mail; o boas-vindas não marca o endereço como verificado e não substitui essa confirmação.

11.6 Banco de dados atualizado

Item	O que mudou
`usuarios.foto_url`	Coluna nova (`varchar(500)`, opcional)
`email_boas_vindas`	Tabela nova, a 19ª: status, tentativas, último erro e data de envio de cada e-mail
`01_schema.sql`, `stock4event_full.sql`	Incluem a coluna e a tabela novas; o dump foi conferido restaurando-o num banco temporário
`02_seed_catalogo.sql`, `data.sql`	Migração idempotente `ADD COLUMN IF NOT EXISTS foto_url`
`01, 02, 03 .sql` e scripts `.sh`	`SET client_encoding = 'UTF8'` e `PGCLIENTENCODING=UTF8`: corrige a leitura dos acentos no Windows
`gerar_er.py`, `stock4event_er.*`	Novo grupo de cor “comunicação”; diagrama refeito do banco real (Figura 5)
`.gitattributes`	`*.sh text eol=lf`: os scripts continuam rodando no Git Bash após um clone no Windows

Bancos já existentes recebem a coluna e a tabela sozinhos ao reiniciar o backend (`ddl-auto=update`). O banco local foi atualizado depois de um backup em `database/backups/`.

11.7 Testes automatizados

Projeto	Comando	Resultado
Backend	`mvn test` (banco `stock4event_test`)	170 testes passando, incluindo perfil da loja, foto de perfil e e-mail de boas-vindas
Frontend	`npx tsc --noEmit` e `npx jest`	Tipos OK; 48 testes passando

- `BoasVindasEmailIT`: fluxo real do cadastro com servidor de e-mail simulado (mock): destinatário e versão corretos, nome escapado, nada enviado em cadastro inválido, duplicado ou desfeito, conta preservada em falha, tentativas simultâneas sem duplicidade e limite de tentativas.
- `BoasVindasEmailTest`: textos, assuntos, URL do botão, imagens por CID, configuração e classificação das falhas.
- Nenhum teste envia e-mail real.

Apêndice — Glossário

Termo	Significado no projeto
Barraca / Quiosque	Ponto de venda em um evento. No banco é “barraca”; nas telas, “quiosque”
Solicitação de compra	Pedido B2B de um comprador para uma distribuidora
Distribuidora / Fornecedor	Empresa do tipo DISTRIBUIDOR ou AMBOS, que aparece no marketplace
Perfil de uso	Cliente, Fornecedor ou ClienteFornecedor — define as telas do app
Perfil (tabela)	Papel de acesso do usuário na empresa (todos são “Admin” hoje)
codigo / codigo_origem	Código único do produto e, no comprador, o código do produto original do fornecedor
DTO	Data Transfer Object: o formato do JSON de entrada/saída
JWT	JSON Web Token: token assinado que identifica o usuário logado
Seed / Runner	Código que popula dados iniciais quando o backend sobe
Polling	O app pergunta ao servidor de tempos em tempos (5 s) se algo mudou
Context (React)	Estado global compartilhado entre telas
AsyncStorage	Armazenamento chave-valor persistente no aparelho

Histórico de versões do documento

Versão	Data	Mudanças
1	21/10/2026	Primeira versão: arquitetura, backend, API, frontend, fluxos e estado do projeto
2	21/10/2026	Banco de dados oficial (capítulo 5), nova identidade visual (capítulo 6), capturas de tela, equipe e instituição, repositório do frontend renomeado, correção do data.sql e roadmap da apresentação
3	22/10/2026	Segurança JWT em todas as rotas, PDV, SaborViva, novo catálogo da Casa dos Vinhos, vitrine entre contas, correção do cancelamento de pedido e banco de dados atualizado (capítulo 10)
4	22/10/2026	Ajustes em subtelas, perfil da loja, foto de perfil, e-mail de boas-vindas, correção da codificação dos scripts do banco e tabela `email_boas_vindas` (capítulo 11)